GRUPO 9 · REQUERIMIENTOS FUNCIONALES

Reportes y Dashboard

Módulo: Panel de control / reportes financieros

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 9
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aiquaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre los dos reportes agregados del sandbox sobre el libro de movimientos (desglose por tipo y resumen general) y, desde la incorporación del CRUD, el alta, consulta, reemplazo y baja de movimientos individuales sobre esa misma tabla.

Fuera de alcance: Exportación a archivo, gráficos, comparación entre períodos, agregados sobre otras tablas (transferencias, pagos, órdenes) y reportes por moneda: los reportes leen únicamente la tabla de movimientos.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy `/api/proxy/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v1/reportes/movimientos	RF-G9-01	Desglose por tipo de movimiento, con cantidad y total, filtrable por titular y por rango de fechas.
GET /api/v1/reportes/resumen	RF-G9-02	Resumen general: cantidad, total y fechas del primer y del último movimiento.
GET /api/v1/movimientos	RF-G9-03	Lista movimientos activos, opcionalmente por titular.
POST /api/v1/movimientos	RF-G9-04	Crea un movimiento individual.
GET /api/v1/movimientos/{id}	RF-G9-05	Devuelve el detalle de un movimiento.
PUT /api/v1/movimientos/{id}	RF-G9-06	Reemplaza un movimiento existente.
DELETE /api/v1/movimientos/{id}	RF-G9-07	Da de baja (soft-delete) un movimiento.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key.".

- Cada API key admite un máximo de **200 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` de la API hace baja lógica (`activo = false`), devolviendo `204 No Content` sin cuerpo.
- Casi todas las tablas de negocio (`sesiones`, `transferencias`, `facturas`, `tarjetas`, `notificaciones`, `ordenes`, `reservas`, `movimientos`, `roles`, además de `usuarios` y `cuentas` que ya la tenían) tienen columna `activo`/`activa`; todo `GET`, `PUT` y `DELETE` de esa tabla filtra u opera solo sobre filas con `activo = true` — una fila dada de baja deja de ser visible y de poder reemplazarse, pero no se borra.
- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.
- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- Los reportes se calculan sobre la tabla `movimientos`, que se carga con los datos sembrados y, desde el CRUD, también con lo que cree `POST /movimientos`.
- Las fechas de los filtros se envían como texto en la URL y las interpreta la base de datos; el formato recomendado es `AAAA-MM-DD` o una marca de tiempo ISO 8601.
- **Los dos reportes (RF-G9-01/02) no se tocaron al agregar el CRUD y no filtran por `activo`:** agregan absolutamente todos los movimientos, incluidos los dados de baja con `DELETE /movimientos/{id}`. Un escenario que dé de baja un movimiento y espere verlo desaparecer del reporte fallará.

3. Modelo de datos del módulo

movimientos

Libro de movimientos que atraviesa los distintos dominios del sandbox. Es la única fuente de los dos reportes.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`tipo_movimiento`	`text`, uno de `transferencia` / `pago_factura` / `compra_ecommerce` / `cargo_tarjeta`
`monto`	`numeric(14,2)`, obligatorio
`referencia_id`	`bigint`, opcional y deliberadamente **sin** clave foránea: según el tipo, apunta a una transferencia, un pago, una orden o una tarjeta
`descripcion`	`text`, opcional
`created_at`	`timestamptz`, por defecto `now()`
`activo`	`boolean`, obligatorio, por defecto `true`; lo usan `GET/PUT/DELETE` del CRUD (RF-G9-03/06/07) para filtrar/operar — **los reportes lo ignoran**

> La tabla de movimientos sigue sin alimentarse automáticamente: registrar una transferencia, pagar una factura o cerrar una compra **no** agrega un movimiento. Con el CRUD, un movimiento puede crearse a mano con `POST /movimientos` (RF-G9-04), pero eso tampoco enlaza con la operación real: `referenciaId` es un número libre que nadie valida contra la tabla que dice referenciar.

4. Requerimientos funcionales

RF-G9-01 — Reporte de movimientos por tipo

`GET /api/v1/reportes/movimientos`

Devuelve, para cada tipo de movimiento, cuántos movimientos hay y cuál es su importe acumulado, sobre el universo que definen los filtros aplicados.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.
- `desde` (opcional, por query): fecha de inicio del período; incluye los movimientos con fecha igual o posterior.
- `hasta` (opcional, por query): fecha de fin del período; incluye los movimientos con fecha igual o anterior.

Reglas de negocio

- Los tres filtros son opcionales y se combinan con Y lógico; sin ninguno, el reporte abarca todos los movimientos del sandbox.
- El resultado agrupa por `tipo_movimiento` y devuelve una fila por tipo presente en el universo filtrado, ordenada alfabéticamente por tipo.
- Un tipo de movimiento sin registros en el período **no aparece** con cantidad cero: simplemente no figura en el resultado.
- La cantidad se devuelve como número entero y el total como valor numérico exacto (texto con decimales).
- **Las fechas no se validan en la entrada:** viajan como texto y las interpreta la base de datos. Un texto no interpretable produce un error de ejecución.
- Tampoco se valida la coherencia del rango: enviar `desde` posterior a `hasta` es aceptado y devuelve un resultado vacío.
- No hay límite de filas ni paginación, porque la cantidad de tipos posibles es acotada.
- **Agrega también los movimientos dados de baja (`activo = false`)** con `DELETE /movimientos/{id}`: este reporte no filtra por esa columna.

Respuesta esperada

- `200 OK` con `data` como arreglo de filas, cada una con `tipo_movimiento`, `cantidad` y `total`.
- Si ningún movimiento cumple los filtros, `data` es un arreglo vacío.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` presente pero no numérico, cero o negativo.
`EXECUTION_ERROR`	400	`desde` o `hasta` con un texto que la base no puede interpretar como fecha.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/reportes/movimientos/route.ts`

RF-G9-02 — Resumen general de movimientos

`GET /api/v1/reportes/resumen`

Devuelve los indicadores del panel de control: cuántos movimientos hay, cuánto suman, y las fechas del primero y del último.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.

Reglas de negocio

- Con `usuarioId`, los indicadores se calculan sobre los movimientos de ese titular; sin él, sobre todos los movimientos del sandbox.
- Devuelve siempre un único objeto, nunca un arreglo.
- Si no hay movimientos que cumplan el filtro, `cantidad_movimientos` es cero, `total` es cero, y las fechas del primero y del último quedan vacías.
- El total se devuelve como cero (y no vacío) cuando no hay movimientos, para que el panel pueda mostrarlo sin conversiones.
- Este reporte no acepta filtros de fecha: siempre abarca toda la historia del titular.
- **Agrega también los movimientos dados de baja**, por la misma razón que RF-G9-01: no filtra por `activo`.

Respuesta esperada

- `200 OK` con `data` conteniendo `cantidad_movimientos` (entero), `total` (numérico), `primero` (fecha del movimiento más antiguo) y `ultimo` (fecha del más reciente).

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` presente pero no numérico, cero o negativo.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/reportes/resumen/route.ts`

RF-G9-03 — Listar movimientos

`GET /api/v1/movimientos`

Devuelve los movimientos individuales del sandbox, con la posibilidad de acotar el resultado a un titular. Es el recurso CRUD genérico sobre la misma tabla que agregan los reportes de arriba.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.

Reglas de negocio

- Devuelve únicamente movimientos con `activo = true`; uno dado de baja con RF-G9-07 deja de aparecer aquí (aunque siga contando en los reportes).
- Con `usuarioId`, devuelve todos los movimientos activos de ese titular ordenados por identificador ascendente; sin filtro, los primeros 100 del sandbox.

Respuesta esperada

- `200 OK` con `data` como arreglo de movimientos.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` presente pero no numérico, cero o negativo.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/movimientos/route.ts`

RF-G9-04 — Crear un movimiento

`POST /api/v1/movimientos`

Registra un movimiento individual en el libro, con su tipo, importe y una referencia opcional a la operación que lo originó.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo.
- `tipoMovimiento` (obligatorio): uno de `transferencia`, `pago_factura`, `compra_ecommerce`, `cargo_tarjeta`.
- `monto` (obligatorio): número mayor que cero.
- `referenciaId` (opcional): número entero positivo.
- `descripcion` (opcional): texto libre.

Reglas de negocio

- `referenciaId` **no tiene clave foránea**: acepta cualquier número entero positivo, exista o no como identificador en la tabla que el `tipoMovimiento` sugiere (transferencias, pagos, órdenes o tarjetas). No hay forma de que la base rechace una referencia inventada.
- El titular debe existir: un `usuarioId` inexistente falla como error de ejecución por violación de clave foránea.
- El movimiento creado aparece de inmediato en los reportes de RF-G9-01/02, sumando a su `tipo_movimiento`.

Respuesta esperada

- `201 Created` con `data` conteniendo el movimiento creado.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, `tipoMovimiento` tiene un valor fuera de la lista permitida, o el `usuarioId` indicado no existe.

Fuente: `app/api/v1/movimientos/route.ts`

RF-G9-05 — Consultar un movimiento

`GET /api/v1/movimientos/{id}`

Devuelve el detalle de un movimiento puntual.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve el movimiento cuyo identificador coincide exactamente, siempre que siga `activo = true`.
- Si no existe o ya está dado de baja, responde 404 con el mensaje "Movimiento no encontrado.".

Respuesta esperada

- `200 OK` con `data` conteniendo el movimiento completo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un movimiento vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/movimientos/[id]/route.ts`

RF-G9-06 — Reemplazar un movimiento

`PUT /api/v1/movimientos/{id}`

Reemplaza el tipo, importe, referencia y descripción de un movimiento existente.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `tipoMovimiento` (obligatorio): uno de `transferencia`, `pago_factura`, `compra_ecommerce`, `cargo_tarjeta`.
- `monto` (obligatorio): número mayor que cero.
- `referenciaId` (opcional): número entero positivo.
- `descripcion` (opcional): texto libre.

Reglas de negocio

- Solo puede reemplazar un movimiento con `activo = true`; uno dado de baja responde 404.
- `usuarioId` no forma parte del cuerpo: el titular del movimiento no puede cambiarse por este reemplazo.
- El nuevo `monto` se refleja de inmediato en los reportes de RF-G9-01/02.

Respuesta esperada

- `200 OK` con `data` conteniendo el movimiento ya actualizado.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un movimiento vigente con ese identificador.
`VALIDATION_ERROR`	400	Falta un campo obligatorio o tiene un valor fuera de la lista permitida.

Fuente: `app/api/v1/movimientos/[id]/route.ts`

RF-G9-07 — Dar de baja un movimiento

`DELETE /api/v1/movimientos/{id}`

Da de baja lógica un movimiento, quitándolo de los listados y consultas por id del CRUD. **No lo quita de los reportes**: RF-G9-01 y RF-G9-02 no filtran por `activo` y lo siguen sumando.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activo = false`; la fila permanece en la base, nunca se borra físicamente.
- Es la única baja del sandbox cuyo efecto es visible en el CRUD pero invisible en los reportes que agregan la misma tabla — una inconsistencia real, no un comportamiento a validar como correcto.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un movimiento vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/movimientos/[id]/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.
- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body.".
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`, ej. un email o número de factura repetido) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos cae en `400 EXECUTION_ERROR`.
- **`DELETE` es siempre baja lógica:** actualiza `activo = false` (o `activa` en `cuentas`) y responde `204 No Content` **sin cuerpo** — no `200` con la fila afectada.

- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera Next.js automáticamente, ninguna ruta lo arma a mano, y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear`, `activar`, `confirmar`, `cancelar`, `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.
- **Las transiciones de estado anteriores al CRUD no miran `activo`:** `PATCH .../bloquear`, `.../activar`, `.../leer`, `.../confirmar`, `.../cancelar`, `.../kyc` y `POST .../pagar` no filtran por `activo = true` (ninguno de esos archivos se tocó al agregar el CRUD). Un recurso dado de baja con el nuevo `DELETE` deja de aparecer en `GET`, pero **sigue pudiendo bloquearse, confirmarse, leerse o pagarse** a través de su acción dedicada — una inconsistencia real del sandbox, no un comportamiento a asumir como intencional.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso; también un POST de upsert que reactivó una fila existente en vez de crearla.
`201 Created`	POST que crea una fila nueva.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Falta el header `x-api-key`, o la clave no existe o está inactiva.
`RATE_LIMITED`	429	Se superaron las 200 requests por minuto de esa API key.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe, ya está dado de baja (`activo=false`), o no está en un estado que admita la operación.
`CONFLICT`	409	Violación de restricción `UNIQUE` en Postgres (ej. email, número de factura o nombre de rol duplicado).
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local. **La interfaz web todavía no tiene pantallas para `PUT` / `DELETE` ni para los recursos nuevos (`sesiones` , `movimientos`)**: solo cubre los flujos de creación y consulta que ya existían; esos requerimientos se prueban llamando la API directamente (Postman, `curl`, o el cliente HTTP del framework de automatización), no navegando la web.

Pantalla	Qué permite hacer
`/reportes` — Panel de control	Una sola pantalla con filtros de titular, fecha desde y fecha hasta, la tarjeta de indicadores del resumen y la tabla del desglose por tipo de movimiento. Ejecuta RF-G9-01 y RF-G9-02.

- Los filtros de fecha aplican solo al desglose por tipo: el resumen general no los usa, porque el endpoint no los acepta.
- Con filtros que no arrojan resultados, la pantalla muestra el estado vacío en lugar de una tabla sin filas.
- No hay pantalla para el CRUD de movimientos: `GET` / `POST /movimientos` y `GET` / `PUT` / `DELETE /movimientos/{id}` (RF-G9-03 a 07) solo se pueden ejercitar llamando la API directamente.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G9-01 — Reporte de movimientos por tipo

- Al pedir el reporte sin filtros, la respuesta es 200 y cada elemento de `data` tiene `tipo_movimiento`, `cantidad` y `total`.
- Los tipos devueltos están ordenados alfabéticamente y no se repiten.
- Al filtrar por un titular, la suma de las cantidades del reporte coincide con la cantidad de movimientos de ese titular.
- Al acotar el período a un rango sin movimientos, la respuesta es 200 con `data` vacío.
- Al enviar `desde` posterior a `hasta`, la respuesta es 200 con `data` vacío, no un error.
- Al enviar `desde=ayer`, la respuesta es 400 `EXECUTION_ERROR`, porque la validación de fechas la hace la base de datos.
- Registrar una transferencia nueva no cambia el resultado del reporte: el libro de movimientos no se alimenta automáticamente.
- Dar de baja un movimiento con RF-G9-07 **no** lo saca de este reporte: sigue sumando en su `tipo_movimiento`.

RF-G9-02 — Resumen general de movimientos

- Al pedir el resumen sin filtro, la respuesta es 200 y `data` es un objeto con las cuatro propiedades del indicador.

- Al pedir el resumen de un titular con movimientos, `data.cantidad_movimientos` coincide con la suma de las cantidades del reporte por tipo del mismo titular.
- Al pedir el resumen de un titular sin movimientos, la respuesta es 200 con `cantidad_movimientos` en cero, `total` en cero y las fechas vacías.
- La fecha de `primero` nunca es posterior a la de `ultimo`.
- Al enviar `usuarioId=0`, la respuesta es 400 `VALIDATION_ERROR`.

RF-G9-03 — Listar movimientos

- Al listar movimientos filtrando por un titular, todos los elementos devueltos tienen ese `usuario_id`.
- Un movimiento dado de baja con RF-G9-07 deja de aparecer en este listado, aunque siga sumando en RF-G9-01/02.

RF-G9-04 — Crear un movimiento

- Al crear un movimiento con datos válidos, la respuesta es 201 y el movimiento aparece en el listado de RF-G9-03.
- Tras crearlo, el reporte de RF-G9-01 filtrado por ese titular incluye el nuevo movimiento en su `tipo_movimiento`.
- Al crear con un `referenciaId` que no corresponde a ninguna fila real, la operación es aceptada igual: no hay validación cruzada.
- Al crear con `tipoMovimiento=deposito`, la respuesta es 400 `VALIDATION_ERROR`.
- Al crear para un `usuarioId` inexistente, la respuesta es 400 `VALIDATION_ERROR`.

RF-G9-05 — Consultar un movimiento

- Dado un movimiento creado con RF-G9-04, al consultarlo por su `id` la respuesta es 200 y los datos coinciden con los enviados.
- Al consultar un movimiento inexistente o dado de baja, la respuesta es 404 `NOT_FOUND`.

RF-G9-06 — Reemplazar un movimiento

- Dado un movimiento existente, al reemplazar su `monto` la respuesta es 200 y el reporte de RF-G9-01 refleja el nuevo importe.
- Al reemplazar un movimiento inexistente o dado de baja, la respuesta es 404 `NOT_FOUND`.

RF-G9-07 — Dar de baja un movimiento

- Dado un movimiento vigente, al darlo de baja la respuesta es 204 sin cuerpo.
- Tras la baja, `GET /movimientos/{id}` responde 404 y el movimiento deja de aparecer en RF-G9-03.
- Tras la baja, el mismo movimiento **sigue sumando** en el reporte de RF-G9-01 y en el resumen de RF-G9-02 para ese titular.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Marco del BCP sobre información de operaciones al cliente	El cliente debe poder consultar el detalle y el resumen de sus operaciones por período.	Parcial: existe el desglose y el resumen, pero el resumen no admite acotar el período y el detalle operación por operación no se expone.
Ley 1334/98 de Defensa del Consumidor (SEDECO), derecho a la información	La información debe ser cierta, clara y suficiente para que el consumidor entienda su actividad.	Parcial: los indicadores son claros, pero el libro de movimientos no refleja automáticamente las operaciones que el propio usuario realiza en el sandbox, y un movimiento dado de baja sigue apareciendo en los totales.
Integridad y trazabilidad de la información contable	Los agregados deben poder reconciliarse contra los registros de origen que los componen, y una baja debe excluirse de los totales.	No implementado: `referencia_id` no tiene clave foránea, no hay endpoint que permita ir del movimiento a la operación que lo originó, y dar de baja un movimiento no lo excluye de los reportes que lo agregan.
Ley 6534/2020 de protección de datos personales	Los reportes de un titular solo deben ser accesibles para ese titular.	No implementado: sin filtro, los reportes agregan la actividad de todos los titulares del sandbox.
Presentación de importes en moneda	Los totales deben expresarse indicando la moneda a la que corresponden.	No implementado: `movimientos` no tiene columna de moneda; los totales suman importes sin distinguirla.

Brechas conocidas del sandbox

- El libro de movimientos sigue sin alimentarse automáticamente con las operaciones del sandbox: transferencias, pagos y órdenes no generan movimientos por sí solas.
- **Los reportes (RF-G9-01/02) no filtran por `activo`: un movimiento dado de baja con `DELETE /movimientos/{id}` desaparece del CRUD pero sigue sumando en ambos reportes.** Es la inconsistencia más notable introducida por el CRUD y la más fácil de pasar por alto al automatizar escenarios.
- `referenciaId` no tiene clave foránea ni en el reporte ni en el CRUD: acepta cualquier número, exista o no la fila referenciada.
- Los filtros de fecha de los reportes no se validan en la entrada; un texto inválido llega hasta la base y se reporta como error de ejecución.
- El resumen no acepta rango de fechas, por lo que no puede compararse un período contra otro.
- Un tipo de movimiento sin registros no aparece con cantidad cero, lo que obliga al consumidor del reporte a completar los faltantes.
- No hay columna de moneda: los importes se suman sin distinguirla.

- No hay exportación ni paginación en los reportes.